

Experiment/Practical 6 Decision Tree

Title: Implementation of Decision Tree for Classification

Aim: To implement and understand the Decision Tree algorithm for classification tasks and evaluate its performance on a given dataset.

Objective: Students will learn

- Understand the decision tree classification mechanism.
 - Implement the decision tree algorithm on a dataset for classification.
 - Evaluate the performance using relevant metrics such as accuracy, precision, recall, and F1-score.
 - Visualize the decision tree structure and its decision boundaries.
-

Problem statement

Use the given dataset(s) to demonstrate the application of the Decision Tree algorithm for classification. The task is to classify the data points into different classes based on the features and to understand how the decision tree algorithm splits the data at each node.

Explanation/Stepwise Procedure/ Algorithm:

- Give a brief description of the decision tree

A decision tree is a graphical representation of different options for solving a problem and show how different factors are related. It has a hierarchical tree structure starts with one main question at the top called a node which further branches out into different possible outcomes where:

1. Root Node is the starting point that represents the entire dataset.
2. Branches: These are the lines that connect nodes. It shows the flow from one decision to another.
3. Internal Nodes are Points where decisions are made based on the input features.
4. Leaf Nodes: These are the terminal nodes at the end of branches that represent final outcomes or predictions

- Give mathematical formulation of decision tree (Entropy, Information Gain, and Gini Index/Impurity)

It is a measure of impurity (non-homogeneity) widely used in decision trees. It aims to measure the probability of misclassifying a randomly chosen element from the dataset. The greater the value of the Gini Index, the greater the chances of having misclassifications.

The Gini Index is calculated using the formula:

$$Gini(t) = 1 - \sum_j [p(j|t)]^2$$

where $p(j|t)$ is the relative frequency of class j at node t .

- The maximum value is $(1 - 1/n)$ indicating that n classes are equally distributed.
- The minimum value is 0 indicating that all records belong to a single class.

Entropy is another measure of impurity, and it is used to quantify the state of disorder, randomness, or uncertainty within a set of data. In the context of decision trees, like the Gini Index, it helps in determining how a node should be split to result in sub-nodes that are as pure (homogenous) as possible.

The Entropy of a node is calculated using the formula:

$$Entropy(t) = - \sum_j p(j|t) \times \log_2(p(j|t))$$

where $p(j|t)$ is the relative frequency of class j at node t .

- The maximum value is $\log_2(n)$ which indicates high uncertainty i.e., n classes are equally distributed.
- The minimum value is 0 which indicates low uncertainty i.e., all records belong to a single class.

Information Gain's objective is similar to that of the Gini Split – it aims to determine the best feature for splitting the data at each node. It does this by calculating the reduction in entropy after a node is split into sub-nodes using a particular feature. The feature with the highest information gain is chosen for the node.

The Information Gain is calculated using the formula:

$$\text{Information Gain} = \text{Entropy}(\text{Parent Node}) - \text{Average Entropy}(\text{Children})$$

where

Average Entropy(Children) equals $\sum_{i=1}^k \frac{n_i}{n} Entropy(i)$

- n_i represents the number of records at child/sub-node i .
- n represents the number of records at the parent node.
- Write the importance of the decision tree

Easy to understand and interpret – Mimics human decision-making, making it simple to follow.

Handles both numerical and categorical data – Works with all types of inputs without complex preprocessing

No need for feature scaling – Unlike other algorithms, decision trees don't require normalization or standardization.

Automatic feature selection – Identifies the most important attributes for decision-making.

Works for both classification and regression – Can be used for predicting categories (spam or not) and numerical values (house prices).

Fast and efficient for small to medium datasets – Can make quick predictions with minimal computing power.

Robust to missing values – Can handle datasets with incomplete information without requiring imputation.

Captures non-linear relationships – Unlike linear models, decision trees can split data in complex ways.

- Brief explanation of performance metrics (e.g., accuracy, precision, recall, f1-score, confusion matrix, cross-validation)

Accuracy

Accuracy is the proportion of all classifications that were correct, whether positive or negative. It is mathematically defined as:

$$\text{Accuracy} = \frac{\text{correct classifications}}{\text{total classifications}} = \frac{TP + TN}{TP + TN + FP + FN}$$

In the spam classification example, accuracy measures the fraction of all emails correctly classified.

Recall, or true positive rate

The true positive rate (TPR), or the proportion of all actual positives that were classified correctly as positives, is also known as recall.

Recall is mathematically defined as:

$$\text{Recall (or TPR)} = \frac{\text{correctly classified actual positives}}{\text{all actual positives}} = \frac{TP}{TP + FN}$$

False positive rate

The false positive rate (FPR) is the proportion of all actual negatives that were classified incorrectly as positives, also known as the probability of false alarm. It is mathematically defined as:

$$\text{FPR} = \frac{\text{incorrectly classified actual negatives}}{\text{all actual negatives}} = \frac{FP}{FP + TN}$$

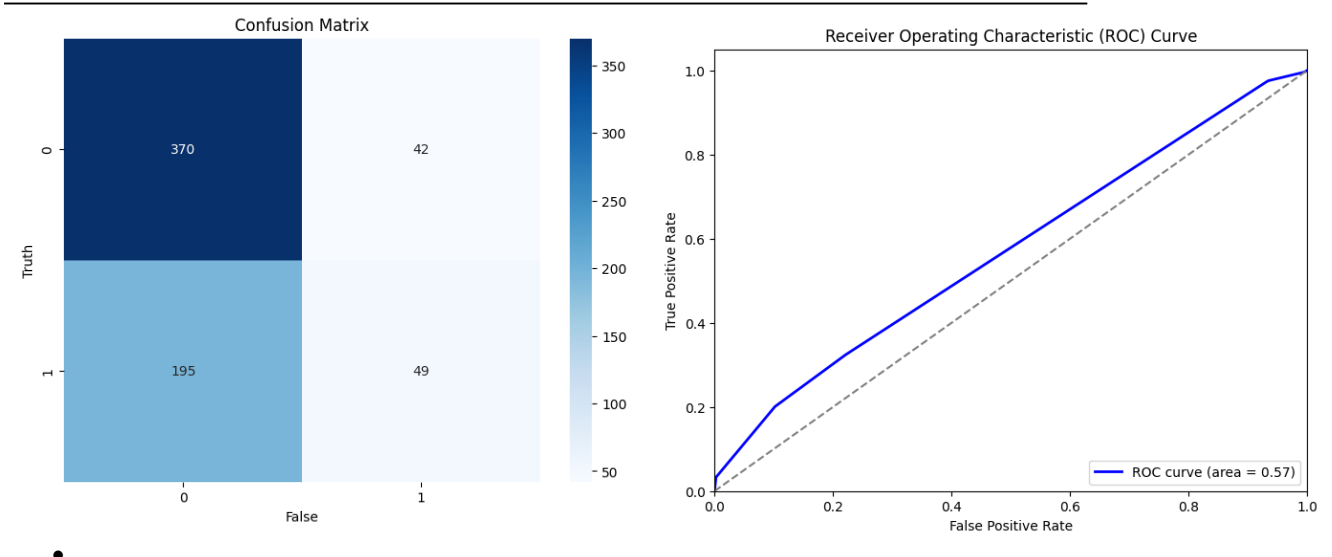
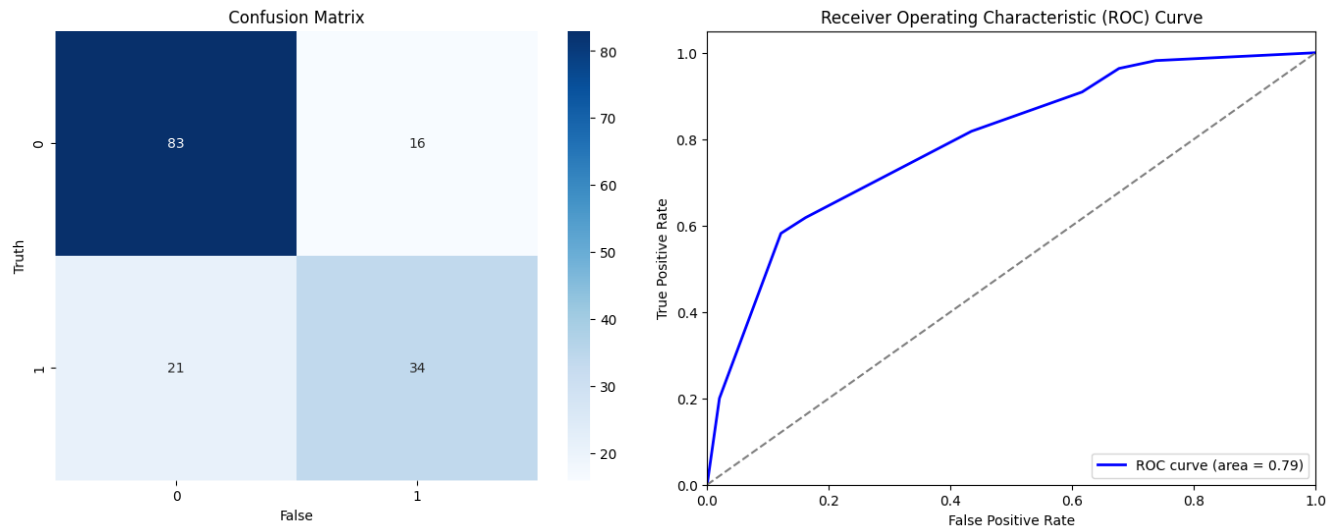
Precision

Precision is the proportion of all the model's positive classifications that are actually positive. It is mathematically defined as:

$$\text{Precision} = \frac{\text{correctly classified actual positives}}{\text{everything classified as positive}} = \frac{TP}{TP + FP}$$

In the spam classification example, precision measures the fraction of emails classified as spam that were actually spam.

- **Add necessary figure(s)/Diagram(s)**



Conclusion:

Impact of hyperparameters:

- **max_depth** – Controls the maximum depth of the decision tree. A higher value allows the tree to learn more complex patterns but may lead to overfitting, while a lower value helps prevent overfitting but might underfit the data.
- **min_samples_split** – Defines the minimum number of samples required to split an internal node. Higher values reduce model complexity and prevent overfitting, while lower values allow deeper trees with more splits.
- **min_samples_leaf** – Specifies the minimum number of samples required in a leaf node. Higher values make the tree more generalized by preventing small, isolated leaf nodes, while lower values allow more granular splits that may overfit the training data.